

Calcolabilità e complessità

Alessandro Carella

Indice

1	Automi e linguaggi	2
1.1	Automi finiti	2
1.2	Non determinismo	6
1.3	Espressioni regolari	14
1.3.1	Definizione formale di espressione regolare	14
1.3.2	Equivalenza con gli automi finiti	16
1.4	Linguaggi non regolari	22
1.4.1	Il pumping lemma per i linguaggi regolari	22

Capitolo 1

Automi e linguaggi

1.1 Automi finiti

Gli automi finiti sono un buon modello per computer una quantità estremamente di memoria che permette di fare molte cose utili. Il sistema di controllo per una porta automatica è un esempio di tale dispositivo.

Gli automi finiti e la loro controparte probabilistica, le *catene di Markov* sono utili strumenti quando cerchiamo di riconoscere pattern in dati. Questi dispositivi sono usati nell'elaborazione vocale e nel riconoscimento ottico dei caratteri. Le catene di Markov sono state perfino usate per modellare e predire i cambiamenti dei prezzi nei mercati finanziari.

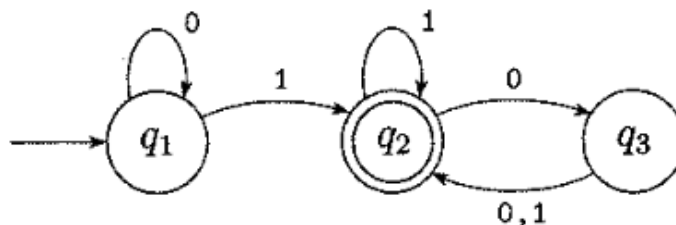
Definizione formale di automa finito

DEFINIZIONE 1.1: AUTOMA FINITO

Un *automa finito* è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove:

1. Q è un insieme finito chiamato l'insieme degli *stati*;
2. Σ è un insieme finito chiamato *alfabeto*;
3. $\delta : Q \times \Sigma \rightarrow Q$ è la *funzione di transizione*;
4. $q_0 \in Q$ è lo *stato iniziale*;
5. $F \subseteq Q$ è l'*insieme degli stati accettanti*.

Consideriamo il seguente automa M_1 :



Possiamo descrivere M_1 formalmente ponendo $M_1 = (Q, \Sigma, \delta, q_1, F)$ ove:

1. $Q = \{q_1, q_2, q_3\}$;
2. $\Sigma = \{0, 1\}$;
3. δ è descritta come segue:

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

4. q_1 è lo stato iniziale;
5. $F = \{q_2\}$

Se A è l'insieme di tutte le stringhe che la macchina M accetta, diciamo che A è il *linguaggio della macchina* M e scriviamo $L(M) = A$. Diciamo che M *riconosce* A o che M *accetta* A . Poichè il termine *accetta* ha significati diversi quando ci riferiamo a macchine che accettano stringhe e macchine che accettano linguaggi, per evitare confusione preferiamo per i linguaggi il termine *riconosce*.

Una macchina può accettare diverse stringhe, ma riconosce sempre soltanto un linguaggio. Se la macchina non accetta alcuna stringa, essa riconosce ancora un linguaggio, il linguaggio vuoto $\emptyset$.

Nel nostro esempio, sia:

$$A = \{w \mid w \text{ contiene almeno un } 1 \text{ e un numero pari di } 0 \text{ segue l'ultimo } 1\}$$

Definizione formale di computazione

Finora abbiamo descritto gli automi finiti informalmente, usando diagrammi di stato, e con una definizione formale, come una quintupla. La descrizione informale è più facile da comprendere all'inizio, ma la definizione formale è più utile per rendere la notazione precisa, risolvendo ogni ambiguità.

Facciamo lo stesso per la computazione di un automa finito.

Sia $M = (Q, \Sigma, \delta, q_0, F)$ un automa finito e sia $w = w_1 w_2 \cdots w_n$ una stringa dove ogni w_i è un elemento dell'alfabeto Σ . Allora M accetta w se esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$;
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ per $i = 0, \dots, n-1$;
3. $r_n \in F$.

La prima condizione afferma che la macchina inizia nello stato iniziale. La seconda condizione afferma che la macchina passa da stato a stato in base alla funzione di transizione. La terza condizione afferma che la macchina accetta il suo input se termina la lettura in uno stato accettante. Diciamo che M riconosce il linguaggio A se $A = \{w \mid M \text{ accetta } w\}$.

DEFINIZIONE 1.2: LINGUAGGIO REGOLARE

Un linguaggio è chiamato un *linguaggio regolare* se un automa finito lo riconosce.

Le operazioni regolari

Dopo aver introdotto e definito gli automi finiti e i linguaggi regolari, iniziamo a studiarne le proprietà. In aritmetica, gli oggetti di base sono i numeri e gli strumenti sono le operazioni per trattarli, come $+$ e $\times$. Nella teoria della computazione, gli oggetti sono i linguaggi e gli strumenti includono operazioni specificatamente progettate per trattarli. Definiamo tre operazioni sui linguaggi, chiamate *operazioni regolari*, e le usiamo per studiare le proprietà dei linguaggi regolari.

DEFINIZIONE 1.3: OPERAZIONI REGOLARI

Siano A e B linguaggi. Definiamo le operazioni regolari *unione*, *concatenazione* e *star* come segue:

- **Unione:** $A \cup B = \{x \mid x \in A \text{ o } x \in B\}$
- **Concatenazione:** $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$

- **Star:** $A^* = \{x_1x_2\dots x_k \mid k \geq 0 \text{ e ogni } x_i \in A\}$

Esempio

Sia Σ l'alfabeto standard a 26 lettere $\{a, b, \dots, z\}$. Se $A = \{good, bad\}$ e $B = \{boy, girl\}$, allora:

$$A \cup B = \{good, bad, boy, girl\}$$

$$A \circ B = \{goodboy, goodgirl, badboy, badgirl\}$$

$$A^* = \{\epsilon, good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, \dots\}$$

Sia $\mathbb{N} = \{1, 2, 3, \dots\}$ l'insieme dei numeri naturali. Quando diciamo che $\mathbb{N}$ è *chiuso rispetto alla moltiplicazione*, intendiamo che per ogni x e per ogni y in $\mathbb{N}$, il prodotto $x \times y \in \mathbb{N}$. Invece $\mathbb{N}$ non è chiuso rispetto alla divisione, poichè 1 e 2 sono in $\mathbb{N}$ ma $1/2 \notin \mathbb{N}$.

In generale, una classe di oggetti è *chiusa* rispetto a un'operazione se l'applicazione di questa operazioni a elementi della classe restituisce un oggetto ancora nella classe. Mostriamo che la classe dei linguaggi regolari è chiusa rispetto a tutte e tre le operazioni regolari.

Teorema 1.1: Chiusura unione

La classe dei linguaggi regolari è chiusa rispetto all'operazioni di unione.

In altre parole, se A_1 e A_2 sono linguaggi regolari, lo è anche $A_1 \cup A_2$.

Quindi, abbiamo due linguaggi regolari, A_1 e A_2 , e vogliamo mostrare che anche $A_1 \cup A_2$ è regolare. Poichè A_1 e A_2 sono regolari, sappiamo che sono riconosciuti da degli automi finiti. Quindi per provare che $A_1 \cup A_2$ è regolare, dobbiamo esibire un automa finito, M , che riconosce $A_1 \cup A_2$.

Dimostrazione. Supponiamo che M_1 riconosca A_1 , dove $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ ed M_2 riconosca A_2 , dove $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$. Costruiamo M che riconosce $A_1 \cup A_2$ dove $M = (Q, \Sigma, \delta, q_0, F)$.

1. $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ e } r_2 \in Q_2\}$.
Questo insieme è il *prodotto cartesiano* degli insiemi Q_1 e Q_2 ed è denotato con $Q_1 \times Q_2$.
2. Σ , l'alfabeto è lo stesso di M_1 e M_2 . In questo teorema e in quelli successivi, assumiamo per semplicità che sia M_1 che M_2 abbiano stesso alfabeto di simboli. Tuttavia, se così non fosse, $\Sigma = \Sigma_1 \cup \Sigma_2$.

3. δ , la funzione di transizione, è definita come segue. Per ogni $(r_1, r_2) \in Q$ e ogni $a \in \Sigma$, sia

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

4. q_0 è la coppia (q_1, q_2) .
5. F è l'insieme delle coppie in cui l'uno o l'altro elemento è uno stato accettante di M_1 o M_2 . Possiamo scriverlo come

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ o } r_2 \in F_2\}$$

Questa espressione è uguale a $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

□

Abbiamo mostrato che l'unione di due linguaggi regolari è regolare, provando che la classe dei linguaggi regolari è chiusa rispetto all'operazione di unione. Ora mostriamo che la classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

Teorema 1.2: Chiusura concatenazione

La classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

In altre parole, se A_1 e A_2 sono linguaggi regolari, allora lo è anche $A_1 \circ A_2$.

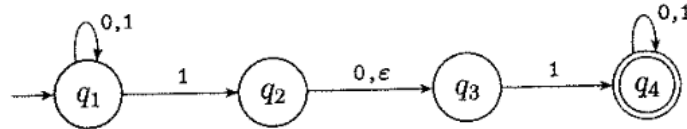
Per provare questo teorema, possiamo iniziare con gli automi finiti M_1 ed M_2 che riconoscono i linguaggi regolari A_1 ed A_2 . Ma ora, invece di costruire M che accetta l'input se M_1 o M_2 lo accetta, esso deve accettare se il suo input può essere diviso in due parti, tali che M_1 accetta la prima parte ed M_2 accetta la seconda parte. Il problema è che M non sa dividere il suo input. Per risolvere questo problema, introduciamo una tecnica chiamata *non determinismo*.

1.2 Non determinismo

Negli esempi visti precedentemente, ogni passo di una computazione seguiva univocamente dal passo precedente. Quando la macchina è in un dato stato e legge il simbolo di input successivo, sappiamo qual è lo stato successivo, esso è univocamente determinato. In questo caso parliamo di computazione *deterministica*. In una macchina *non deterministica*, possono esistere diverse scelte per lo

stato successivo in ogni punto.

Il non determinismo è una generalizzazione del determinismo, quindi ogni automa finito deterministico è automaticamente un automa finito non deterministico. Consideriamo il seguente automa:



La differenza tra un automa finito deterministico, DFA, e un automa finito non deterministico, indicato con NFA, è evidente. In primo luogo, ogni stato di un DFA ha sempre esattamente un arco uscente per ogni simbolo nell'alfabeto. L'NFA viola questa regola, infatti, lo stato q_1 dell'automata della figura precedente, ha un arco uscente per 0, ma ne ha due per 1. In un NFA, uno stato può avere zero, uno, o più archi uscenti per ogni simbolo dell'alfabeto.

Inoltre, in un DFA, le etichette sugli archi di transizione sono simboli dell'alfabeto. Questo NFA ha un arco con etichetta ϵ .

Durante la computazione di un NFA, quando si giunge ad uno stato con più modi per procedere, la macchina si divide in più copie di sè stessa e segue tutte le possibilità in parallelo. Infine, se una qualunque di queste copie della macchina è in uno stato accettante alla fine dell'input, l'NFA accetta la stringa di input.

Il non determinismo può essere visto come una specie di computazione parallela dove "processi" multipli indipendenti o "threads" possono essere eseguiti contemporaneamente. Un altro modo per considerare una computazione non deterministica è vederla come un albero di possibilità. La radice dell'albero corrisponde all'inizio della computazione. Considerando sempre l'automata precedente, l'albero di computazione è il seguente:

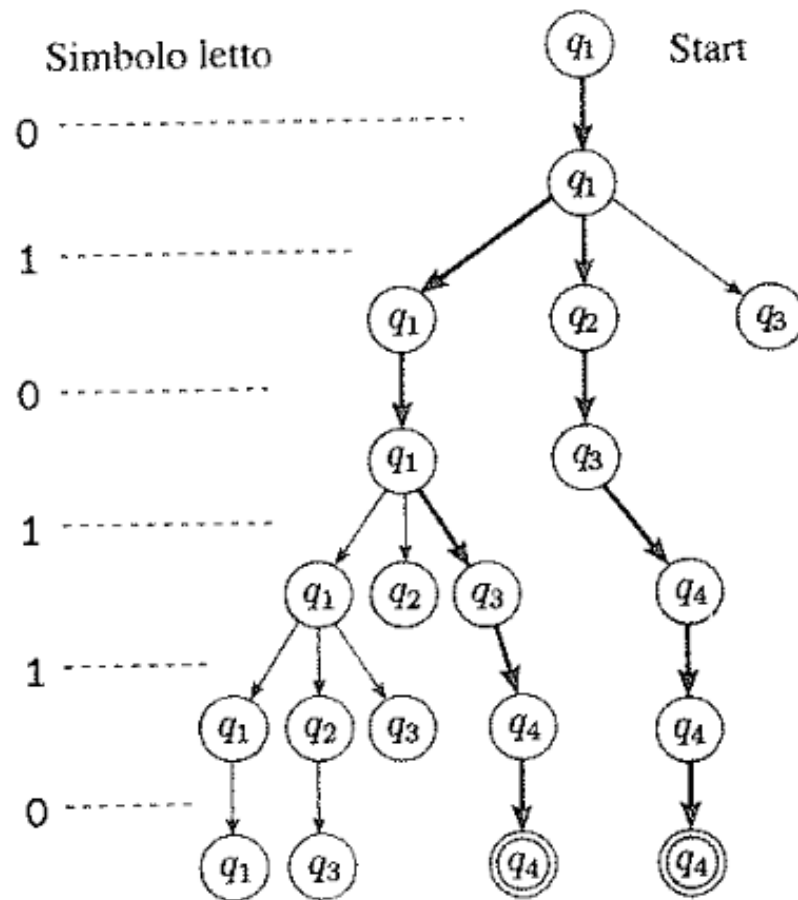


Figura 1.1: La computazione dell'automa precedente su 010110

Definizione formale di automa finito non deterministico

La definizione formale di automa finito non deterministico è simile a quella di automa finito deterministico. Essi differiscono nel tipo di funzione di transizione. In un DFA, la funzione di transizione prende uno stato e un simbolo di input e produce lo stato successivo. In un NFA, la funzione di transizione prende uno stato e un simbolo di input o *la stringa vuota* e produce l'*insieme dei possibili stati successivi*.

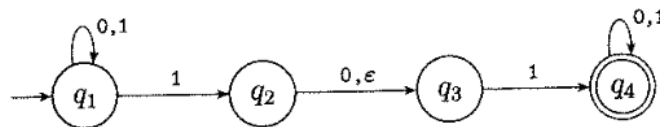
DEFINIZIONE 1.4: AUTOMA FINITO NON DETERMINISTICO

Un *automa finito non deterministico* è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove

1. Q è un insieme finito di stati,
2. Σ è un alfabeto finito,
3. $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ è la funzione di transizione,
4. $q_0 \in Q$ è lo stato iniziale,
5. $F \subseteq Q$ è l'insieme degli stati di accettazione.

Esempio

Riconsideriamo il seguente NFA N_1 :



La descrizione formale di N_1 è $(Q, \Sigma, \delta, q_1, F)$ dove

1. $Q = \{q_1, q_2, q_3, q_4\}$
2. $\Sigma = \{0, 1\}$
3. δ è data da

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

4. q_1 è lo stato iniziale
5. $F = \{q_4\}$

La definizione formale di computazione per un NFA è simile a quella per un DFA. Sia $N = (Q, \Sigma, \delta, q_0, F)$ un NFA e sia w una stringa sull'alfabeto Σ . Diciamo

che N accetta w se possiamo scrivere w come $w = y_1 y_2 \dots y_m$, dove ciascun y_i è un elemento di $\sum_\epsilon$ ed esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, y_{i+1})$ per $i = 0, \dots, m-1$
3. $r_m \in F$

Equivalenza tra gli NFA e i DFA Gli automi finiti deterministici e quelli non deterministici riconoscono la stessa classe di linguaggi. Diciamo che due macchine sono *equivalenti* se esse riconoscono lo stesso linguaggio.

Teorema 1.3: Equivalenza NFA e DFA

Per ogni automa finito non deterministico esiste un automa finito deterministico equivalente.

Se un linguaggio è riconosciuto da un NFA, allora dobbiamo mostrare l'esistenza di un DFA che lo riconosce. L'idea è trasformare l'NFA in un DFA equivalente che simula l'NFA.

Dimostrazione. Sia $N = (Q, \sum, \delta, q_0, F)$ l'NFA che riconosce un linguaggio A . Costruiamo un DFA $M = (Q', \sum, \delta', q'_0, F')$ che riconosce A . Prima di fare la costruzione completa, consideriamo inizialmente il caso più semplice in cui N non ha ϵ -archi.

1. $Q' = \mathcal{P}(Q)$
Ogni stato di M è un insieme di stati di N .
2. Per $R \in Q'$ e $a \in \sum$, sia $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a)\}$ per qualche $r \in R$
Se R è uno stato di M , esso è anche un insieme di stati di N . Quando M legge un simbolo a nello stato R . Poichè da ogni stato si può andare in un insieme di stati, prendiamo l'unione di tutti questi insiemi. Un altro modo per scrivere quest'espressione è:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

3. $q'_0 = \{q_0\}$

4. $F' = \{R \in Q' \mid R \text{ contiene uno stato accettante di } N\}$

Ora dobbiamo considerare gli ϵ -archi. Per farlo introduciamo qualche ulteriore notazione. Per ogni stato R di M , definiamo $E(R)$ come la collezione di stati che possono essere raggiunti dagli elementi di R proseguendo solo con ϵ -archi, includendo gli stessi elementi di R . Formalmente, per $R \subseteq Q$ sia

$$E(R) = \{q \mid q \text{ può essere raggiunto da } R \text{ attraverso 0 o più } \epsilon\text{-archi}\}$$

Modifichiamo la funzione di transizione di M ponendo le dita su tutti gli stati che possono essere raggiunti proseguendo attraverso ϵ -archi dopo ogni passo. Sostituendo $\delta(r, a)$ con $E(\delta(r, a))$ realizziamo questo effetto. La funzione di transizione è la seguente.

$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ per qualche } r \in R\}$$

Inoltre, dobbiamo modificare lo stato iniziale di M per muovere inizialmente le dita su tutti i possibili stati che possono essere raggiunti dallo stato iniziale di N attraverso gli ϵ -archi. Cambiare q'_0 in $E(\{q_0\})$ realizza questo risultato. Abbiamo così completato la costruzione del DFA M che simula l'NFA N . $\square$

Corollario 1.1

Un linguaggio è regolare se e solo se qualche automa finito non deterministico lo riconosce.

Una direzione della condizione “se e solo se” afferma che un linguaggio è regolare se qualche NFA lo riconosce. Tuttavia, abbiamo dimostrato che ogni NFA può essere trasformato in un DFA. Conseguentemente, se un NFA riconosce un linguaggio, lo fa anche qualche DFA, e quindi il linguaggio regolare. L'altra direzione della condizione “se e solo se” afferma che un linguaggio è regolare solo se qualche NFA lo riconosce. Questa condizione è vera perchè un linguaggio regolare ha un DFA che lo riconosce e un DFA è anche un NFA.

Chiusura rispetto alle operazioni regolari

Torniamo alla chiusura della classe dei linguaggi regolari rispetto alle operazioni regolari. Dobbiamo provare che l'unione, la concatenazione e lo star di linguaggi regolari sono ancora regolari.

Teorema 1.4

La classe dei linguaggi regolari è chiusa rispetto all'operazione di unione.

Abbiamo i linguaggi regolari A_1 ed A_2 e vogliamo provare che $A_1 \cup A_2$ è regolare. L'idea è provare che due NFA, N_1 ed N_2 per A_1 ed A_2 , e comporli in un nuovo NFA, N . La macchina N deve accettare il suo input se N_1 o N_2 accetta questo input. La nuova macchina ha uno stato iniziale che si dirama negli stati iniziali delle vecchie macchine con ϵ archi.

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere $A_1 \cup A_2$.

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$
2. Lo stato q_0 è lo stato iniziale di N
3. L'insieme degli stati accettanti è $F = F_1 \cup F_2$
4. Definiamo δ in modo che per ogni $q \in Q$ e $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

□

Teorema 1.5: L

classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

L'idea è prendere due NFA, N_1 ed N_2 , per A_1 ed A_2 , e combinarli in un nuovo NFA N . Poniamo come stato iniziale di N lo stato iniziale N_1 . Gli stati accettanti di N_1 hanno degli ulteriori ϵ -archi che non deterministicamente permettono di diramarsi in N_2 ogni volta che N_1 è in uno stato accettore, indicando che ha trovato un pezzo iniziale dell'input che costituisce una stringa in A_1 . Gli stati accettandi di N sono gli stati accettanti di N_2 .

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_1, F_2)$ per riconoscere $A_1 \circ A_2$.

1. $Q = Q_1 \cup Q_2$

2. Lo stato q_1 è uguale allo stato iniziale di N_1
3. Gli stati accettanti F_2 sono uguale agli stati accettanti di N_2
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \sum_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ e } a = \epsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

□

Teorema 1.6

La classe dei linguaggi regolari è chiusa rispetto all'operazione star.

Prendiamo un NFA N_1 per A_1 e lo modifichiamo per riconoscere A_1^* . L'NFA N risultante accetterà il suo input quando esso può essere diviso in varie parti ed N_1 accetta ogni parte. Possiamo costruire N come N_1 con ϵ -archi supplementari che dagli stati accettanti ritornano allo stato iniziale. In questo modo, quando l'elaborazione giunge alla fine di una parte che N_1 accetta, la macchina N ha la scelta di tornare indietro allo stato iniziale per provare a leggere un'altra parte che N_1 accetta. Inoltre, dobbiamo modificare N in modo che accetti ϵ , che è sempre un elemento di A_1^* .

Dimostrazione. Sia $N_1 = (Q_1, \sum, \delta_1, q_1, F_1)$ che riconosce A_1 . Costruiamo $N = (Q, \sum, \delta, q_0, F)$ per riconoscere A_1^* .

1. $Q = \{q_0\} \cup Q_1$
2. Lo stato q_0 è il nuovo stato iniziale.
3. $F = \{q_0\} \cup F_1$
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \sum_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(qa) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ e } a = \epsilon \\ \{q_1\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$



1.3 Espressioni regolari

Possiamo usare le operazioni regolari per costruire espressioni che descrivono linguaggi, che sono chiamate *espressioni regolari*. Ad esempio:

$$(0 \cup 1)0^*$$

Il valore di un'espressione regolare è un linguaggio. In questo caso, il valore è il linguaggio che consiste di tutte le stringhe che iniziano con 0 o un 1 seguito da un qualsiasi numero di simboli uguali a 0.

1.3.1 Definizione formale di espressione regolare

DEFINIZIONE 1.5: ESPRESSIONE REGOLARE

Diciamo che R è un'espressione regolare se R è

1. a per qualche a nell'alfabeto Σ ;
2. ϵ ;
3. $\emptyset$;
4. $(R_1 \cup R_2)$ dove R_1 ed R_2 sono espressioni regolari;
5. $(R_1 \circ R_2)$ dove R_1 ed R_2 sono espressioni regolari;
6. $(R_1)^*$ dove R_1 è un'espressione regolare.

È bene non confondere le espressioni regolari ϵ e $\emptyset$. L'espressione regolare ϵ rappresenta il linguaggio che contiene la sola stringa vuota, mentre $\emptyset$ rappresenta il linguaggio che non contiene alcuna stringa.

Inoltre, per comodità, usiamo R^+ come abbreviazione per RR^* e possiamo scrivere R^* come $R^+ \cup \epsilon$.

Quando vogliamo distinguere un'espressione regolare R e il linguaggio che descrive, denotiamo con $L(R)$ il linguaggio di R .

Analizziamo alcune proprietà.

1. $a^* \emptyset = \emptyset$
Concatenando l'insieme vuoto a un qualsiasi insieme si ottiene l'insieme vuoto.
2. $\emptyset^* = \{\epsilon\}$
L'operazione star concatena un numero qualsiasi di stringhe del linguaggio per ottenere una stringa nel risultato. Se il linguaggio è vuoto, l'operazione star può concatenare 0 stringhe, dando solo la stringa vuota.
3. $R \cup \emptyset = R$
Aggiungere il linguaggio vuoto a un qualsiasi altro linguaggio non lo cambia.
4. $R \circ \epsilon = R$
Concatenare la stringa vuota a una qualsiasi stringa non la cambia.

Tuttavia, scambiare $\emptyset$ e ϵ nelle precedenti identità può far sì che le uguaglianze non siano vere.

1. $R \cup \epsilon$ può non essere uguale a R
Per esempio, se $R = \emptyset$, allora $L(R) = \{\emptyset\}$ ma $L(R \cup \epsilon) = \{\emptyset, \epsilon\}$.
2. $R \circ \emptyset$ può non essere uguale a R .
Per esempio, se $R = \emptyset$, allora $L(R) = \{\emptyset\}$ ma $L(R \circ \emptyset) = \emptyset$

Le espressioni regolari sono un utile strumento nel progetto dei compilatori per i linguaggi di programmazione. Gli oggetti elementari in un linguaggio di programmazione, chiamati *token*, come i nomi delle variabili e le costanti, possono essere descritti con espressioni regolari. Ad esempio, una costante numerica può avere una parte decimale e/o un segno e può essere descritta come un elemento del linguaggio

$$(+ \cup - \cup \epsilon)(D^+ \cup D^+.D^* \cup D^*.D^+)$$

Una volta che la sintassi di un linguaggio di programmazione è stata descritta con un'espressione regolare in termini dei suoi token, dei sistemi automatici possono generare l'*analizzatore lessicale*, la parte di un compilatore che elabora inizialmente il programma input.

1.3.2 Equivalenza con gli automi finiti

Le espressioni regolari e gli automi finiti sono equivalenti rispetto alla loro potenza descrittiva. Ogni espressione regolare può essere trasformata in un automa finito che riconosce il linguaggio che essa descrive, e viceversa.

Teorema 1.7

Un linguaggio è regolare se e solo se qualche espressione regolare lo descrive.

Questo teorema deve essere dimostrato in entrambe le direzioni. Enunciamo e proviamo ciascuna direzione in un lemma separato.

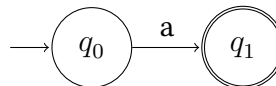
LEMMA 1.1

Se un linguaggio è descritto da un'espressione regolare, allora esso è regolare.

Supponiamo di avere un'espressione regolare R che descrive un linguaggio A . Mostriamo come trasformare R in un NFA che riconosce A .

Dimostrazione. Trasformiamo R in un NFA N . Consideriamo i sei casi nella definizione di **espressione regolare**.

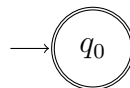
1. $R = a$ per qualche $a \in \Sigma$. Allora $L(R) = \{a\}$ e il seguente NFA riconosce $L(R)$.



Questa macchina soddisfa la definizione di NFA ma non quella di DFA perchè ha qualche stato con nessun arco uscente per ogni possibile simbolo di input.

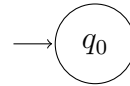
Formalmente, $N = (\{q_0, q_1\}, \Sigma, \sigma, q_0, \{q_1\})$ dove descriviamo σ dicendo che $\sigma(q_0, a) = \{q_1\}$ e $\sigma(r, b) = \emptyset$ per $r \neq q_0$ o $b \neq a$.

2. $R = \epsilon$. Allora $L(R) = \{\epsilon\}$ e il seguente NFA riconosce $L(R)$.



Formalmente $N = (\{q_0\}, \Sigma, \sigma, q_0, \{q_0\})$, dove $\sigma(r, b) = \emptyset$ per ogni r e b .

3. $R = \emptyset$. Allora $L(R) = \emptyset$ e il seguente NFA riconosce $L(R)$.



Formalmente, $N = (\{q_0\}, \Sigma, \sigma, q_0, \emptyset)$ dove $\sigma(r, b) = \emptyset$ per ogni r e b .

4. $R = R_1 \cup R_2$
 5. $R = R_1 \circ R_2$
 6. $R = R_1^*$

Per gli ultimi tre casi, usiamo le costruzioni date nelle prove che la classe dei linguaggi regolari è chiusa rispetto alle operazioni regolari. $\square$

Questo termina la prima parte del teorema dando la direzione più facile della condizione “se e solo se”.

LEMMA 1.2

Se un linguaggio è regolare, allora è descritto da un'espressione regolare.

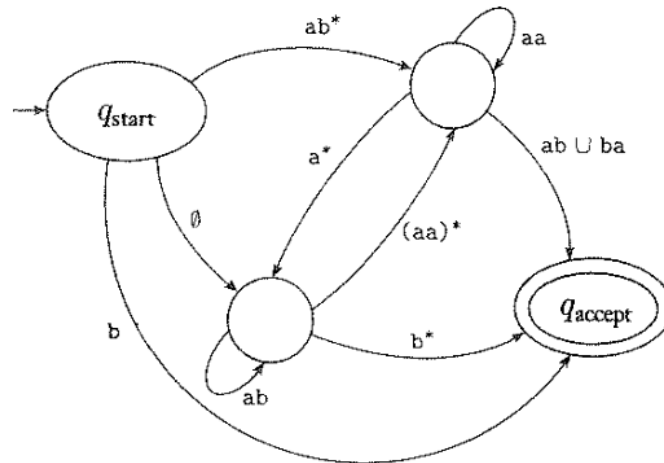
Dobbiamo mostrare che se un linguaggio A è regolare, allora un'espressione regolare lo descrive. Poiché A è regolare, esso è accettato da un DFA. Descriviamo una procedura per trasformare i DFA in espressioni regolari equivalenti.

Dividiamo questa procedura in due parti, usando un nuovo tipo di automa finito chiamato *automa finito non deterministico generalizzato*, GNFA.

Prima mostriamo come trasformare un DFA in un GNFA e poi come trasformare un GNFA in un'espressione regolare.

Gli automi finiti non deterministici generalizzati sono automi finiti non deterministici nei quali gli archi delle transizioni possono avere espressioni regolari come etichette, invece che solo elementi dell'alfabeto o ϵ . Un GNFA è non deterministico e quindi può avere diversi modi di elaborare la stessa stringa di input.

La figura seguente presenta un esempio di GNFA.



Per comodità, richiediamo che i GNFA abbiano sempre una forma speciale che soddisfi le seguenti condizioni.

- Lo stato iniziale ha archi di transizione uscenti verso un qualsiasi altro stato ma nessun arco entrante proveniente da un qualsiasi altro stato.
- Esiste un solo stato accettante, ed esso ha archi entranti provenienti da un qualsiasi altro stato ma nessun arco uscente verso un qualsiasi altro stato. Inoltre, lo stato accettante non è uguale allo stato iniziale.
- Eccetto che per lo stato iniziale e lo stato accettante, un arco va da ogni stato ad ogni altro stato e anche da ogni stato a sé stesso.

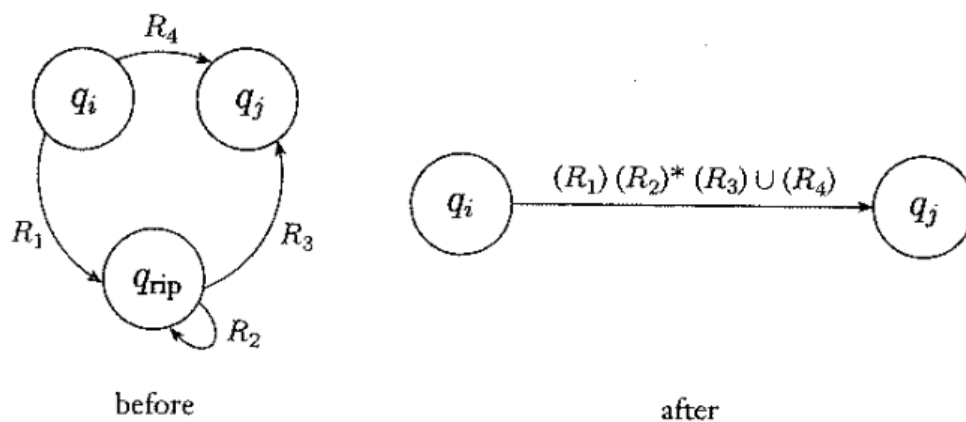
Possiamo facilmente trasformare un DFA in un GNFA. Aggiungiamo semplicemente un nuovo stato iniziale con un ϵ -arco che entra nel vecchio stato iniziale e un nuovo stato accettante con ϵ -archi entranti, provenienti dai vecchi stati accettanti. Se alcuni archi hanno più etichette (o se ci sono più archi che collegano gli stessi due stati nella stessa direzione), sostituiamo ognuno di essi con un solo arco la cui etichetta è l'unione delle precedenti etichette. Infine, aggiungiamo archi con etichetta $\emptyset$ tra stati che non hanno archi. Quest'ultimo non cambierebbe il linguaggio riconosciuto perchè una transizione etichettata con $\emptyset$ non può mai essere usata.

Ora mostriamo come trasformare un GNFA in un'espressione regolare. Supponiamo che il GNFA abbia k stati. Allora, poichè un GNFA deve avere uno stato

iniziale e uno stato accettante ed essi devono essere diversi tra loro, sappiamo che $k \geq 2$. Se $k > 2$, costruiamo un GNFA equivalente con $k - 1$ stati. Questo passo può essere ripetuto sul nuovo GNFA fino a quando esso è ridotto a 2 stati. Se $k = 2$, il GNFA ha un solo arco che va dallo stato iniziale allo stato accettante. L'etichetta di questo arco è l'espressione regolare equivalente.

Il passo cruciale è costruire un GNFA equivalente con uno stato in meno quando $k > 2$. Lo facciamo scegliendo uno stato, estraendolo dalla macchina, e modificando il resto in modo che sia riconosciuto lo stesso linguaggio. Ogni stato può essere scelto, purché non sia lo stato iniziale o lo stato accettante. Chiamiamo q_{rip} lo stato rimosso.

Dopo aver rimosso q_{rip} modifichiamo la macchina cambiando le espressioni regolari che etichettano ciascuno dei restanti archi. Le nuove etichette controbilanciano l'assenza di q_{rip} reintegrando le computazioni perse. La nuova etichetta che va da uno stato q_i a uno stato q_j è un'espressione regolare che descrive tutte le stringhe che porterebbero la macchina da q_i a q_j o direttamente o tramite q_{rip} . Analizziamo tale procedimento nella seguente figura.



Nella vecchia macchina, se

1. q_i va a q_{rip} con un arco etichettato R_1 ;
2. q_{rip} va in sè stesso con un arco etichettato R_2 ;
3. q_{rip} va a q_j con un arco etichettato R_3 ;
4. q_i va a q_j con un arco etichettato R_4 ,

allora nella nuova macchina, l'arco da q_i a q_j riceve l'etichetta

$$(R_1)(R_2)^*(R_3) \cup (R_4)$$

Facciamo questa modifica per ogni arco che va da un qualsiasi stato q_i a un qualsiasi stato q_j , includendo il caso in cui $q_i = q_j$. La nuova macchina riconosce il linguaggio di partenza.

Dimostrazione. Formalizziamo quest'idea. In primo luogo, definiamo formalmente l'automa GNFA. Un GNFA è simile ad un automa finito non deterministico tranne che per la funzione di transizione, che ha la forma

$$\delta : (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \rightarrow R$$

Il simbolo R è la collezione di tutte le espressioni regolari sull'alfabeto Σ e q_{start} e q_{accept} sono gli stati iniziale e accettante.

Il dominio della funzione di transizione è $(Q - \{q_{accept}\}) \times (Q - \{q_{start}\})$ perchè un arco collega ogni stato a un qualsiasi altro stato, tranne che nessun arco è uscente da q_{accept} o è entrante in q_{start} . $\square$

DEFINIZIONE 1.6: AUTOMA FINITO NON DETERMINISTICO GENERALIZZATO

Un *automa finito non deterministico generalizzato* è una quintupla $(Q, \Sigma, \delta, q_{start}, q_{accept})$, dove

1. Q è l'insieme finito degli stati;
2. Σ è l'alfabeto di input;
3. $\delta : (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \rightarrow R$ è la funzione di transizione;
4. q_{start} è lo stato iniziale;
5. q_{accept} è lo stato accettante.

Un GNFA accetta una stringa w in Σ^* se $w = w_1 w_2 \cdots w_k$, dove ogni w_i è in Σ^* ed esiste una sequenza di stati $q_0, q_1, \dots, q_k$ tale che

1. $q_0 = q_{start}$ è lo stato iniziale,
2. $q_k = q_{accept}$ è lo stato accettante e
3. per ogni i , risulta $w_i \in L(R_i)$, dove $R_i = \delta(q_{i-1}, q_i)$

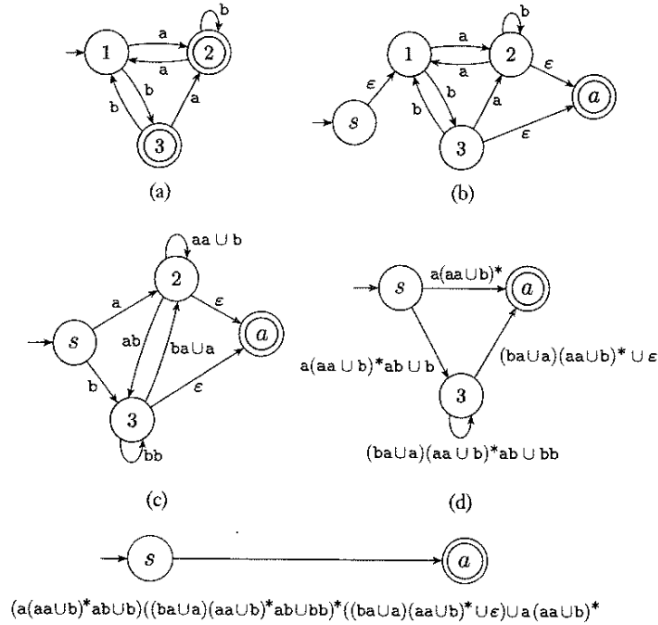
Torniamo alla dimostrazione del **lemma 1.2**, sia M il DFA per il linguaggio A . Allora trasformiamo M in un GNFA G aggiungendo un nuovo stato iniziale un nuovo stato accettante e i necessari archi di transizione supplementari. Usiamo la procedura $\text{CONVERT}(G)$, che prende un GNFA e restituisce un'espressione regolare equivalente. Questa procedura usa la *ricorsione*.

```

1 CONVERT(G) :
2   if k = 2 then return R;
3   if k > 2 then scelto  $q_{rip} \neq q_{accept}$  e  $q_{start}$ 
4     CONVERT(G')
5    $G' = (Q', \Sigma, \delta, q_{start}, q_{accept})$ 
6    $Q' = Q - \{q_{rip}\}$ 
7   foreach  $q_i \in Q' - \{q_{accept}\}$  and  $q_j \in Q' - \{q_{start}\}$ 
8      $\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4)$ 
9      $R_1 = \delta(q_i, q_{rip})$ 
10     $R_2 = \delta(q_{rip}, q_{rip})$ 
11     $R_3 = \delta(q_{rip}, q_j)$ 
12     $R_4 = \delta(q_i, q_j)$ 
13   endfor

```

Analizziamo un esempio, iniziamo con un DFA con tre stati. I passi nella trasformazione sono mostrati nella figura seguente.



1.4 Linguaggi non regolari

In questa sezione, mostriamo come provare che alcuni linguaggi non possono essere riconosciuti da alcun automa finito. Consideriamo il linguaggio $B = \{0^n 1^n \mid n \geq 0\}$. Se cerchiamo di trovare un DFA che riconosca B , scopriamo che la macchina sembra aver bisogno di ricordare quanti simboli uguali a 0 ha visto fin quando legge l'input. Poichè il numero di simboli uguali a 0 non è limitato, la macchina dovrà tenere traccia di un numero infinito di possibilità. Ma non può farlo con un numero finito di stati.

Non è detto che se un linguaggio sembra richiedere memoria non limitata allora è un linguaggio non regolare. Per esempio, consideriamo i due linguaggi sull'alfabeto $\Sigma = \{0, 1\}$:

- $C = \{w \mid w \text{ ha lo stesso numero di simboli uguali a 0 e simboli uguali a 1}\}$
- $D = \{w \mid w \text{ ha un numero uguale di occorrenze di 01 e 10 come sottostringhe}\}$

A prima vista, una macchina che riconosce sembra aver bisogno di contare in ciascun caso,, e quindi nessuno dei due linguaggi sembra essere regolare.

Sappiamo che C non è regolare, ma D è regolare.

1.4.1 Il pumping lemma per i linguaggi regolari

La tecnica per provare la regolarità deriva da un teorema sui linguaggi regolari, chiamato il *pumping lemma*. Questo teorema afferma che tutti i linguaggi regolari hanno una proprietà speciale. Se noi possiamo mostrare che un linguaggio non ha questa proprietà, siamo sicuri che esso non è regolare. La proprietà afferma che tutte le stringhe nel linguaggio possono essere “replicate” se la loro lunghezza raggiunge almeno uno specifico valore speciale, chiamato la *lunghezza del pumping*.

Teorema 1.8: Pumping lemma per linguaggi regolari

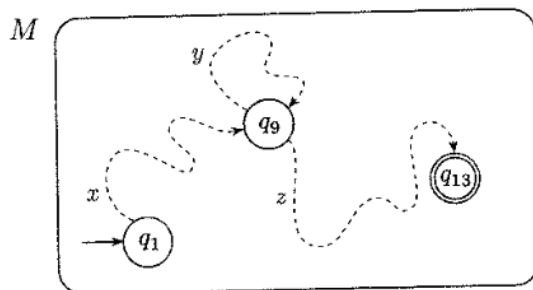
Se A è un linguaggio regolare, allora esiste p (lunghezza del pumping) tale che se s è una qualsiasi stringa in A di lunghezza almeno p , allora s può essere divisa in tre parti, $s = xyz$, soddisfacenti le seguenti condizioni:

1. $\forall i \geq 0, xy^i z \in A$
2. $|y| > 0$
3. $|xy| \leq p$

Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A . Assegniamo alla lunghezza del pumping p il numero degli stati di M . Mostriamo che per ogni stringa s in A di lunghezza almeno p può essere divisa nelle tre parti xyz , soddisfacenti le tre condizioni.

Se $s \in A$ ha lunghezza almeno p , consideriamo la sequenza di stati che M attraversa nella computazione con input s . Inizia con lo stato iniziale q_1 , poi va per esempio in q_3 , poi in q_{20} , e così via, finché raggiunge la fine di s nello stato q_{13} . Se s è in A , sappiamo che M accetta s , quindi q_{13} è uno stato accettante. Se supponiamo che n sia la lunghezza di s , la sequenza di stati $q_1 q_3 q_{20} \dots q_{13}$ ha lunghezza $n + 1$. Poiché n è almeno p , sappiamo che $n + 1 > p$, il numero di stati di M . Quindi, la sequenza deve contenere uno stato che si ripete. Questo risultato è un esempio del *principio della piccionaia*.

Di seguito viene mostrato un esempio di un DFA con uno stato che si ripete.



Ora dividiamo s nelle tre componenti x, y e z . La componente x è la parte di s che compare prima di q_9 , la componente y è la parte tra le due occorrenze di q_9 e la componente z è la parte restante di s , che viene dopo la seconda occorrenza di q_9 .

Vediamo perché questa suddivisione di s soddisfa le tre condizioni. Supponiamo di eseguire M sull'input $xyyz$. Sappiamo che x porta M da q_1 a q_9 , e poi il primo y lo riporta da q_9 a q_9 , come fa il secondo y e poi z lo porta in q_{13} che accetta $xyyz$ in quanto stato accettante. Analogamente, esso accetterà $xy^i z \forall i > 0$.

Per verificare la condizione 2, vediamo che $|y| > 0$, poiché era la parte di s tra due diverse occorrenze dello stato q_9 . Per ottenere la condizione 3, ci assicuriamo che q_9 sia la prima ripetizione nella sequenza. Per il principio della piccionaia, i primi $p + 1$ stati nella sequenza devono contenere una ripetizione. Quindi $|xy| \leq p$.

Dimostrazione. Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A e sia p il numero di stati di M .

Sia $s = s_1 s_2 \dots s_n$ una stringa in A di lunghezza n , dove $n \geq p$. Sia $r_1, \dots, r_{n+1}$ la sequenza di stati attraversati da M mentre elabora s , quindi $r_{i+1} = \delta(r_i, s_i)$ per $1 \leq i \leq n$. Questa sequenza ha lunghezza $n + 1$, che è almeno $p + 1$. Due stati tra i primi $p + 1$ elementi nella sequenza devono essere lo stesso stato, per il principio della piccioni. Chiamiamo il primo di questi r_j e il secondo r_l . Poichè r_l si presenta tra le prime $p + 1$ posizioni in una sequenza che inizia in r_1 , abbiamo $l \leq p + 1$. Ora sia $x = s_1 \dots s_{j-1}$, $y = s_j \dots s_{l-1}$ e $z = s_l \dots s_n$.

Poichè x porta M da r_1 a r_j , y porta M da r_j a r_j e z porta M da r_j a r_{n+1} , che è uno stato accettante, M deve accettare $xy^i z$ per $i \geq 0$. Sappiamo che $j \neq l$, perciò $|y| > 0$ e $l \leq p + 1$, perciò $|xy| \leq p$. Quindi tutte le condizioni del pumping lemma sono rispettate. $\square$

Per usare il pumping lemma per provare che un linguaggio B non è regolare, in primo luogo si assuma che B sia regolare per ottenere una contraddizione. Poi si usi il pumping lemma per assicurare l'esistenza di una lunghezza del pumping p tale che tutte le stringhe di lunghezza maggiore o uguale a p in B possano essere iterate. In seguito, si trovi una stringa s in B che ha una lunghezza maggiore o uguale a p , ma che non può essere iterata. Infine, si dimostri che s non può essere iterata considerando tutti i modi di dividere s in x, y e z e, per ogni tale divisione, trovando un valore i tale che $xy^i z \notin B$.

Esempio

Sia $B = \{0^n 1^n \mid n \geq 0\}$. Usiamo il pumping lemma per provare che B non è regolare.

Assumiamo che B sia regolare. Sia p la lunghezza del pumping data dal pumping lemma. Scegliamo $s = 0^p 1^p$. Poichè $s \in B$ ed $|s| \geq p$, il pumping lemma assicura che s può essere divisa in tre parti, $s = xyz$, tali che per ogni $i \geq 0$ la stringa $xy^i z \in B$. Consideriamo tre casi per mostrare che questo risultato è impossibile.

1. La stringa y consiste solo di simboli uguali a 0. In questo caso, la stringa $xyyz$ ha più simboli uguali a 0 che simboli uguali a 1 e quindi non è un elemento di B , violando la condizione 1 del pumping lemma.
2. La stringa y consiste solo di simboli uguali a 1. Anche questo caso porta ad una contraddizione.

3. La stringa y consiste solo di simboli uguali a 0 che di simboli uguali a 1. In questo caso, la stringa $xyyz$ può avere lo stesso numero di simboli uguali a 0 e simboli uguali a 1, ma essi non saranno nell'ordine corretto, con qualche 1 prima di qualche 0. Quindi non è un elemento di B , il che è una contraddizione.

Pertanto una contraddizione è inevitabile se assumiamo che B sia regolare, quindi B non è regolare.

Esempio

Sia $C = \{w \mid w \text{ ha lo stesso numero di simboli uguali a 0 e simboli uguali a 1}\}$. Usiamo il pumping lemma per mostrare che C non è regolare.

Assumiamo al contrario che C sia regolare. Sia p la lunghezza del pumping data dal pumping lemma. Sia s la stringa $0^p 1^p$. Poichè $s \in C$ che ha lunghezza maggiore di p , il pumping lemma assicura che s può essere divisa in tre parti, $s = xyz$, tali che per ogni $i \geq 0$ la stringa $xy^i z \in C$. Se prendiamo x e z uguali alla stringa vuota e y uguale alla stringa $0^p 1^p$, allora $xy^i z$ ha sempre lo stesso numero di simboli uguali a 0 e simboli uguali a 1 e quindi è in C .

Qui la condizione 3 del pumping lemma è utile. Essa stabilisce che quando iteriamo s , essa deve essere divisa in modo che $|xy| \leq p$. Questa restrizione sul modo in cui s può essere divisa rende più facile mostrare che la stringa $s = 0^p 1^p$ che abbiamo scelto non può essere iterata. Se $|xy| \leq p$, allora y deve consistere solo di simboli uguali a 0, perciò $xyyz \notin C$. Quindi, s non può essere iterata.

Scegliere la stringa s in questo esempio richiede più attenzione. Se avessimo scelto $s = (01)^p$, avremmo avuto dei problemi perchè abbiamo bisogno di una stringa che **non può essere iterata** e quella stringa può essere iterata, perfino prendendo in considerazione la condizione 3.

Un metodo alternativo per provare che C non è regolare segue dal fatto che sappiamo che B non è regolare. Se C fosse regolare, anche $C \cap 0^* 1^*$ sarebbe regolare. Questo perchè il linguaggio $0^* 1^*$ è regolare e la classe dei linguaggi regolari è chiusa rispetto all'intersezione.